

From local commands to verified delivery.

A five-slide walkthrough of what I built, what I proved, and what I would improve.

PRESENTED BY

Mark Andrei Castillo

Local WSL walkthrough | checkpoint evidence

THE STORY

Linux

- > Docker
- > CI/CD
- > Kubernetes HA
- > Part 5 reflection

Rule: local action -> captured result -> checkpoint meaning

First, I checked the machine and made the checks repeatable

SYSTEM_HEALTH.SH - IMPORTANT OUTPUT ONLY

```
draiimon@Aloof:~$ nano devops-exam/scripts/system_health.sh
draiimon@Aloof:~$ chmod +x devops-exam/scripts/system_health.sh
bash devops-exam/scripts/system_health.sh
=====
SYSTEM HEALTH REPORT
=====

[1] DATE & TIME
Thu Aug 6 11:52:56 CST 2026

[2] UPTIME
11:52:56 up 14 min, 1 user, load average: 0.02, 0.23, 0.15

[3] MEMORY USAGE
total      used      free      shared    buff/cache   available
Mem:      1.9Gi      584Mi      96Mi      3.1Mi      1.4Gi      1.4Gi
Swap:      0B          0B          0B

[4] DISK SPACE
Filesystem      Size      Used Avail Use% Mounted on
none            987M      0      987M  0% /usr/lib/modules/5.15.167.4-microsoft-standard-WSL2
none            987M      4.0K    987M  1% /mnt/wsl
drivers          465G      390G      75G  84% /usr/lib/wsl/drivers
/dev/sdb         1007G      2.2G    954G  1% /
none            987M      80K    987M  1% /mnt/wslg
none            987M      0      987M  0% /usr/lib/wsl/lib
rootfs           983M      2.4M    981M  1% /init
none            987M      496K    986M  1% /run
none            987M      0      987M  0% /run/lock
none            987M      0      987M  0% /run/shm
tmpfs            4.0M      0      4.0M  0% /sys/fs/cgroup
none            987M      76K    987M  1% /mnt/wslg/versions.txt
none            987M      76K    987M  1% /mnt/wslg/doc
C:\             465G      390G      75G  84% /mnt/c
D:\             50M       16M      35M  32% /mnt/d
```

LOCAL WSL CAPTURE - ZOOMED

WHAT THIS MEANS

I checked uptime, memory, disk space, and CPU before moving to the next part.

SAY THIS SIMPLY

I used a script so I could check the machine in a repeatable way.

Then, I ran the API, UI, and database together

DOCKER-COMPOSE PS - FINAL HEALTHY STATUS

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
01e90b32318e	ui-app:latest	"docker-entrypoint.s..."	4 minutes ago	Up 4 minutes (healthy)	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp	devops_ui
1fd001b945c7	api-app:latest	"sh -c 'until python..."	4 minutes ago	Up 4 minutes (healthy)	0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp	devops_api
ccfc04243790	mysql:8.0	"docker-entrypoint.s..."	4 minutes ago	Up 4 minutes (healthy)	3306/tcp, 33060/tcp	devops_db

draimon@Aloof:~/devops-exam/part2-docker\$

LOCAL WSL CAPTURE - ZOOMED

WHAT THIS MEANS

The three services were running: API, database, and UI. I checked the application response, not only the image build.

SAY THIS SIMPLY

Docker packages the services. Compose lets me start them together and check if they are healthy.

PART 3 | CI/CD

The pipeline checks the work before publishing images

GITHUB ACTIONS - SUCCESSFUL WORKFLOW

← CI/CD Pipeline - 2026

✓ Add Docker Hub deploy stage #5 Re-run all jobs ...

Summary

All jobs

- ✓ Verify workflow
- ✓ Build Docker images
- ✓ Test applications
- ✓ Push images to Docker Hub

Run details

- Usage
- Workflow file

Triggered via push 5 minutes ago

Status: **Success** Total duration: **4m 14s** Artifacts: -

drailimon pushed → f5d58e1 staging

deploy.yml
on: push

✓ Verify workflow 4s → ✓ Build Docker images 40s → ✓ Test applications 1m 30s → ✓ Push images to Docker ... 1m 39s

Annotations
4 warnings

CHECKPOINTED CI EVIDENCE - ZOOMED

WORKFLOW ORDER

Verify -> Build -> Test -> Push. Each step must pass before the next step continues.

SAY THIS SIMPLY

The pipeline is like a checklist. It tests the application before sending the images to Docker Hub.

I tested availability, then explained the decisions

KUBERNETES PODS - API AND UI REPLICAS READY

CPU:0%MEM:0%

<shift-j> Jump Owner<s>Shell

NAME↑	PF	READY	STATUS	RESTARTS	pods(devops-exam)[5]			MEM	%MEM/R	%MEM/L	IP	NODE	AGE
					CPU	%CPU/R	%CPU/L						
api-app-77c77d88c9-bt9lm	●	1/1	Running	0	2	2	0	77	60	15	10.244.1.8	minikube-m02	6m24s
api-app-77c77d88c9-fb5tl	●	1/1	Running	0	2	2	0	58	45	11	10.244.1.6	minikube-m02	6m47s
db-7ff45cf46-7hmb2	●	1/1	Running	0	6	6	1	444	173	57	10.244.1.3	minikube-m02	19m
ui-app-68f99bcd4-m4rp6	●	1/1	Running	0	1	1	0	50	39	9	10.244.1.9	minikube-m02	5m44s
ui-app-68f99bcd4-vrrwf	●	1/1	Running	0	1	1	0	108	84	21	10.244.1.7	minikube-m02	6m47s

LOCAL WSL CAPTURE - ZOOMED

NODE-FAILURE TEST - 20/20 HTTP 200

[illegible]

LOCAL WSL CAPTURE - ZOOMED

MY SIMPLE TAKEAWAY

I chose Minikube because it was practical for the exam. The important part was testing real traffic. With more time, I would add image scanning, a tested rollback, and production monitoring.

01. Opening and presentation scope

SAY

Good day po. I am Mark Andrei Castillo. I will briefly explain what I built for the Junior DevOps Engineer Exam 2026. I will show the work in order: Linux, Docker, CI/CD, and Kubernetes. For each part, I will say what I did, what result I captured, and what that result means.

SHOW / DO

WALKTHROUGH ACTION

Start with the title slide. Read the title and say: I will keep this as a short walkthrough of my actual work and evidence.

EVIDENCE NOTE

The screenshots came from the documented local work. The repository contains the code, documentation, and evidence used for this presentation.

IF ASKED

If I forget a detail, I will refer to the repository file or screenshot instead of guessing.

02. Parts 1 and 2 foundation

SAY

For Part 1, I started with basic Linux checks. I used a script to check the machine time, memory, disk, and CPU. The purpose was simple: before running an application, I wanted to know the condition of the machine. For Part 2, I used Docker Compose to run the API, UI, and database together. The important result is that the services were running and healthy.

SHOW / DO

WALKTHROUGH ACTION

Point to the large Linux screenshot and say: This is the system health report. Then say: The Docker result shows the services running together.

EVIDENCE NOTE

The screenshot is evidence that the checks were actually run. The Docker stack was corrected before it was treated as ready.

IF ASKED If asked what Docker does, say: Docker packages the application. Compose starts the related services together, so I can test them as one local stack.

03. Part 3 CI/CD

SAY

For Part 3, I connected the staging push to GitHub Actions. The pipeline follows a simple order: verify, build, test, and push. This means the application is checked before the images are published. The screenshot shows the workflow completed successfully.

SHOW / DO

WALKTHROUGH ACTION

Point from left to right and read the job names. Say: The pipeline works like a checklist. Each step must pass before the next step continues.

EVIDENCE NOTE

The successful workflow is captured evidence. I will only claim what the screenshot and repository actually show. Optional rollback and new failure-notification captures are not presented as completed.

IF ASKED

If asked why this is useful, say: It reduces manual steps and catches problems before publishing an image.

04. Part 4 high availability

SAY

For Part 4, I used Kubernetes through a two-node Minikube cluster. I ran more than one copy of the API so the application could continue serving requests if one node had a problem. The screenshots show the pods running and the node-failure test returning 20 out of 20 successful responses.

SHOW / DO

WALKTHROUGH ACTION

Point to the rows marked Running and 1/1. Then point to Successful node-failure requests: 20/20. Say: This is the actual runtime test, not only the configuration file.

EVIDENCE NOTE

The important proof is the captured request result while one node was unavailable. The Kubernetes work and screenshots came from the documented local environment.

IF ASKED If asked what Minikube is, say: It is a local Kubernetes cluster that let me demonstrate the required deployment and availability behavior for the exam.

05. Part 5 decisions and closing

SAY

The main problems I encountered were missing application source, matching the database setup, clean CI runners, and proving that Kubernetes was really serving traffic. I solved these by checking the source, aligning the database, setting up each CI job clearly, and running a runtime test. My main lesson is: do not assume something works just because it looks configured - test the actual result.

SHOW / DO

WALKTHROUGH ACTION

Point to the two large Kubernetes screenshots. Then read the takeaway box. Finish with: Thank you po. I am ready to explain any file or screenshot in more detail.

EVIDENCE NOTE

I will separate completed evidence from future improvements. Image scanning, a tested rollback, and production monitoring are improvements I would add next.

IF ASKED

If I do not know an answer, say: I did not want to guess. I can show the exact file or checkpoint where that behavior is documented.